# Activity 4 Report: Database Users, Roles, Privileges, and Access Control

COMP 018 Database Administration | Project under test: Binary Sisters, Restaurant Online Ordering & Table Reservation System | Date of testing: 6 November 2026

## 1. Cover and links

| Item | Value |
|---|---|
| Our group | G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph |
| Owning group | Binary Sisters (PHP 8 + PostgreSQL) |
| Repository tested | https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System |
| Our repository folder | https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests |
| Tracker | https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests |
| Evidence files | a3_verify_privs.txt, a4_inventory.txt, a4_access_tests.txt, a4_otherdb_tests.txt, a4_audit.txt |

## 2. Projects set up

| Item | Value |
|---|---|
| Database | grp_binarysisters_projectdb (schema public, owner postgres) |
| Setup method | database/setup.ps1 -WithRoles on a local PostgreSQL server, run unchanged |

| Item | Value |
|---|---|
| Setup result | **Success with issues**. The database, tables, seed data and roles were created. The script database/roles/03_set_passwords.psql is missing from the repository, so the login accounts had no passwords (see section 7) |
| Seed data loaded | 51 customers, 10 staff, 50 menu items, 20 tables, 60 orders, 167 order items, 56 payments, 50 reservations |
| Tables (8) | customer, menu_item, order_item, orders, payment, reservation, restaurant_table, staff |
| Views (4) | v_customer_directory, v_order_summary, v_reservation_schedule, v_staff_directory |
| Sequences (7) | One per SERIAL column (order_item has a composite key, so it has none) |
| Extensions | btree_gist 1.8, pgcrypto 1.4, plpgsql |
| Credential columns | customer.password and staff.password (bcrypt hashes) |

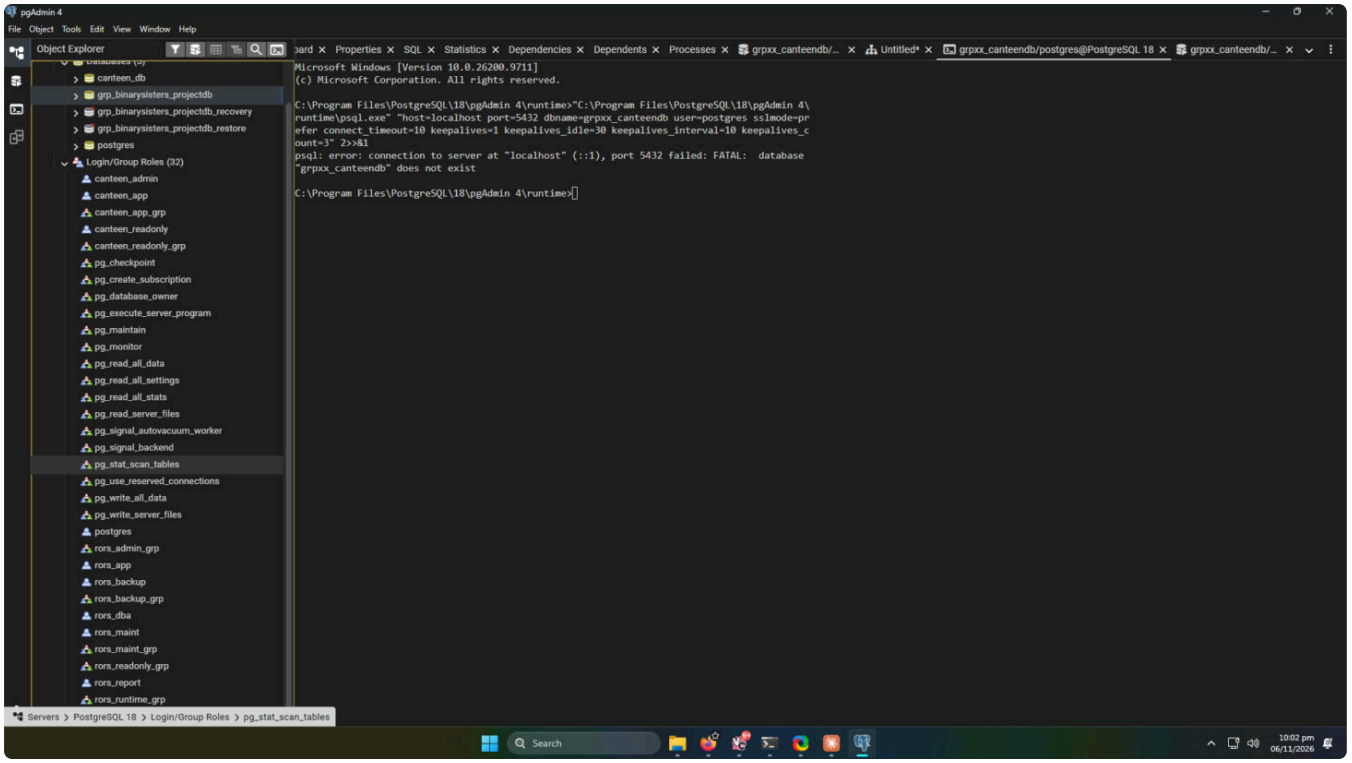
## 3. Access-control plan

The application does not give end users database logins. Customer, Staff and Administrator are rows in the customer and staff tables plus a PHP session, and the web application connects through a single runtime account. The database roles below therefore serve the application runtime and the DBA tasks of Activities 5 and 6.

| Login role | Group role (NOLOGIN) | Role served | Responsibility | Effective privileges (from \dp) |
|---|---|---|---|---|
| rors_app | rors_runtime_grp | Application runtime (Customer, Staff and Administrator sessions) | Normal application operations | SELECT, INSERT, UPDATE on customer, orders, payment. SELECT, INSERT, UPDATE, DELETE on menu_item, reservation, restaurant_table, staff. SELECT, INSERT on order_item. USAGE and SELECT on all sequences. SELECT on v_order_summary and v_reservation_schedule |
| rors_report | rors_readonly_grp | Viewer or auditor | Reporting and inspection | SELECT on menu_item, order_item, orders, payment, reservation, restaurant_table and the four views. No access to customer or staff base tables |
| rors_maint | rors_maint_grp | Maintenance (Activity 5) | VACUUM, ANALYZE, monitoring | pg_monitor and pg_maintain. No table privileges |
| rors_backup | rors_backup_grp | Backup operator (Activity 6) | pg_dump | pg_read_all_data (read only) |
| rors_dba | rors_admin_grp | Database administrator | Day-to-day administration | ALL on public tables and sequences (including TRUNCATE, REFERENCES, TRIGGER), CREATE on schema public, pg_monitor |

Memberships (from the setup output): rors_app to rors_runtime_grp, rors_report to rors_readonly_grp, rors_maint to rors_maint_grp, rors_backup to rors_backup_grp, rors_dba to rors_admin_grp. pg_monitor is granted to rors_maint_grp and rors_admin_grp, pg_maintain to rors_maint_grp, and pg_read_all_data to rors_backup_grp.

Figure 1. pgAdmin Login/Group Roles list showing the ten rors_ roles (five group roles and five login roles) next to the predefined pg_ roles.




### Rationale for the privileges

- The runtime account has no DELETE on orders, order_item and payment, because order history must be kept. Cancelling an order only changes its status. It has no DELETE on customer for the same reason.
- The runtime account has no DDL, no TRUNCATE and no ownership of any object, so a compromised application cannot change the schema.
- The read-only role is kept away from customer and staff so password hashes are never exposed. It reads people only through views that omit the hash.
- The backup role gets read access through the predefined role pg_read_all_data and cannot modify anything.
- The maintenance role has no SELECT on application data. It can inspect activity and run VACUUM and ANALYZE only.
- No role is a superuser and none has CREATEDB or CREATEROLE. Connection limits are set per login: rors_app 20, rors_report 5, rors_maint 3, rors_dba 3, rors_backup 2.

## 4. SQL scripts summary

| Script or command | Purpose | Origin |
|---|---|---|
| database/roles/01_create_roles.sql | Creates group roles, login roles, memberships, predefined-role grants | Owning group, run unchanged |

| Script or command | Purpose | Origin |
|---|---|---|
| database/roles/02_grants.sql | Revokes PUBLIC, grants CONNECT, USAGE, table and sequence privileges, default privileges | Owning group, run unchanged |
| database/roles/04_verify_privileges.sql | Permitted and denied matrix using SET ROLE (26 of 26 PASS) | Owning group, run unchanged |
| ALTER ROLE rors_* PASSWORD (random 16-character values) | Sets passwords that the missing 03_set_passwords.psql should have set. Passwords are stored only in a local file outside the repository | Our group |
| \dt+ \dv \ds \dx \du+ \dp \ddp | Inventory and privilege listing (a4_inventory.txt) | Our group |
| Real-login test script (24 statements) | Permitted and denied tests using each actual account over a real connection (a4_access_tests.txt) | Our group |
| Other-database connection tests | Connection attempts to postgres and template1 as rors_app (a4_otherdb_tests.txt) | Our group |
| Audit queries on pg_roles, role_table_grants, pg_default_acl | Elevated attributes, PUBLIC grants, default privileges (a4_audit.txt) | Our group |

The repository folder is https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests. No passwords or .env files are committed.

## 5. Verification results

The owning group's own matrix uses SET ROLE inside rolled-back transactions and passed 26 of 26 tests. It cannot test login or the CONNECT privilege, so we repeated the important cases as real logins with each account. All 26 results below matched the expected outcome.

| # | Account | Operation | Expected | Actual |
|---|---|---|---|---|
| 1 | rors_app | SELECT count(*) FROM orders | Allowed | Allowed (61 rows) |
| 2 | rors_app | SELECT count(*) FROM customer | Allowed | Allowed (51 rows) |
| 3 | rors_app | INSERT INTO menu_item (inside a rolled-back transaction) | Allowed | Allowed, sequence value 53 returned |
| 4 | rors_app | DELETE FROM orders | Denied | permission denied for table orders |
| 5 | rors_app | DELETE FROM payment | Denied | permission denied for table payment |
| 6 | rors_app | DELETE FROM order_item | Denied | permission denied for table order_item |
| 7 | rors_app | TRUNCATE menu_item | Denied | permission denied for table menu_item |
| 8 | rors_app | DROP TABLE menu_item | Denied | must be owner of table menu_item |
| 9 | rors_app | CREATE TABLE public.zz | Denied | permission denied for schema public |
| 10 | rors_app | CREATE DATABASE | Denied | permission denied to create database |
| 11 | rors_app | CREATE ROLE | Denied | permission denied to create role |
| 12 | rors_report | SELECT count(*) FROM orders | Allowed | Allowed (61 rows) |
| 13 | rors_report | SELECT count(*) FROM v_order_summary | Allowed | Allowed (61 rows) |
| 14 | rors_report | SELECT password FROM customer | Denied | permission denied for table customer |
| 15 | rors_report | SELECT password FROM staff | Denied | permission denied for table staff |
| 16 | rors_report | UPDATE menu_item | Denied | permission denied for table menu_item |

| # | Account | Operation | Expected | Actual |
|---|---|---|---|---|
| 17 | rors_report | INSERT INTO menu_item | Denied | permission denied for table menu_item |
| 18 | rors_report | DELETE FROM menu_item | Denied | permission denied for table menu_item |
| 19 | rors_backup | SELECT count(*) FROM customer | Allowed | Allowed (51 rows) |
| 20 | rors_backup | UPDATE customer | Denied | permission denied for table customer |
| 21 | rors_maint | SELECT count(*) FROM orders | Denied | permission denied for table orders |
| 22 | rors_maint | SELECT count(*) FROM pg_stat_activity | Allowed | Allowed (9 sessions visible) |
| 23 | rors_dba | CREATE ROLE | Denied | permission denied to create role |
| 24 | rors_dba | CREATE DATABASE | Denied | permission denied to create database |
| 25 | rors_app | Connect to database postgres | Denied | FATAL: permission denied for database postgres (no CONNECT privilege) |
| 26 | rors_app | Connect to database template1 | Denied | FATAL: permission denied for database template1 (no CONNECT privilege) |

**Audit results (a4_audit.txt and a3_verify_privs.txt)**

- None of the ten rors_ roles is a superuser, and none has CREATEDB or CREATEROLE.
- No privileges remain granted to PUBLIC on the tables in schema public (0 rows).
- Default privileges exist only for objects created by postgres in schema public, and they go to rors_admin_grp (tables arwdDxtm, sequences rwU). New tables are not exposed to the runtime or read-only groups automatically.
- Row counts later in the session (Activity 6 verification) are higher than the seed data: orders 61 (seed 60), order_item 169 (167), payment 57 (56), reservation 52 (50) and menu_item 51 (50). The pattern fits one order with two dishes, its payment and two

reservations created through the application during the Activity 3 click-through. The group confirmed they were made by hand through the application.

## 6. Application check under the least-privilege account

The application was configured with DB_USER=rors_app and tested for Customer, Staff and Administrator workflows. Result: pass. A customer order (#65) was placed and listed, Staff could open the order and payment pages but received the Access denied page on /admin/staff, and the Administrator could open the Staff accounts page. No database permission denied message appeared on any captured screen, so the grants cover what the code needs. The full click-through screenshots are in the Activity 3 report.

Figure 2. Customer order #65 placed through the application running as rors_app.

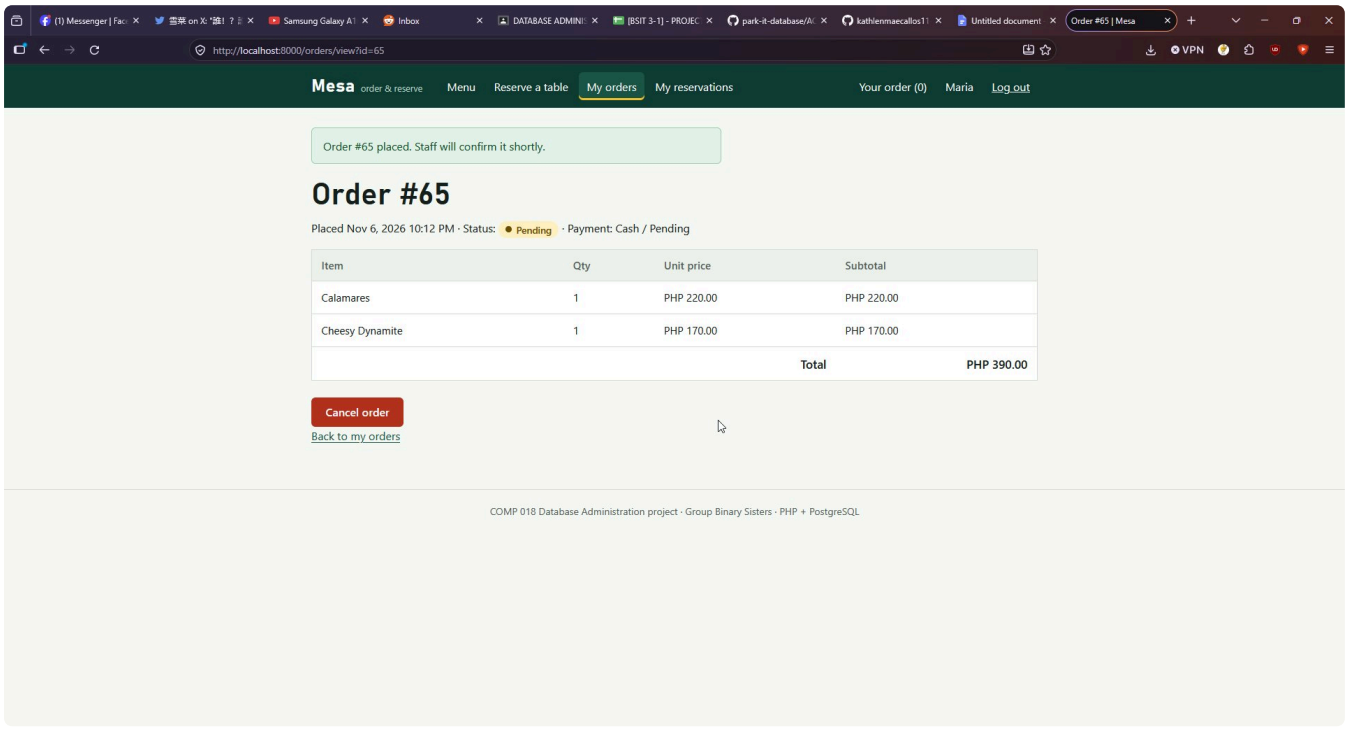



Figure 3. Staff role opening /admin/staff and receiving Access denied.

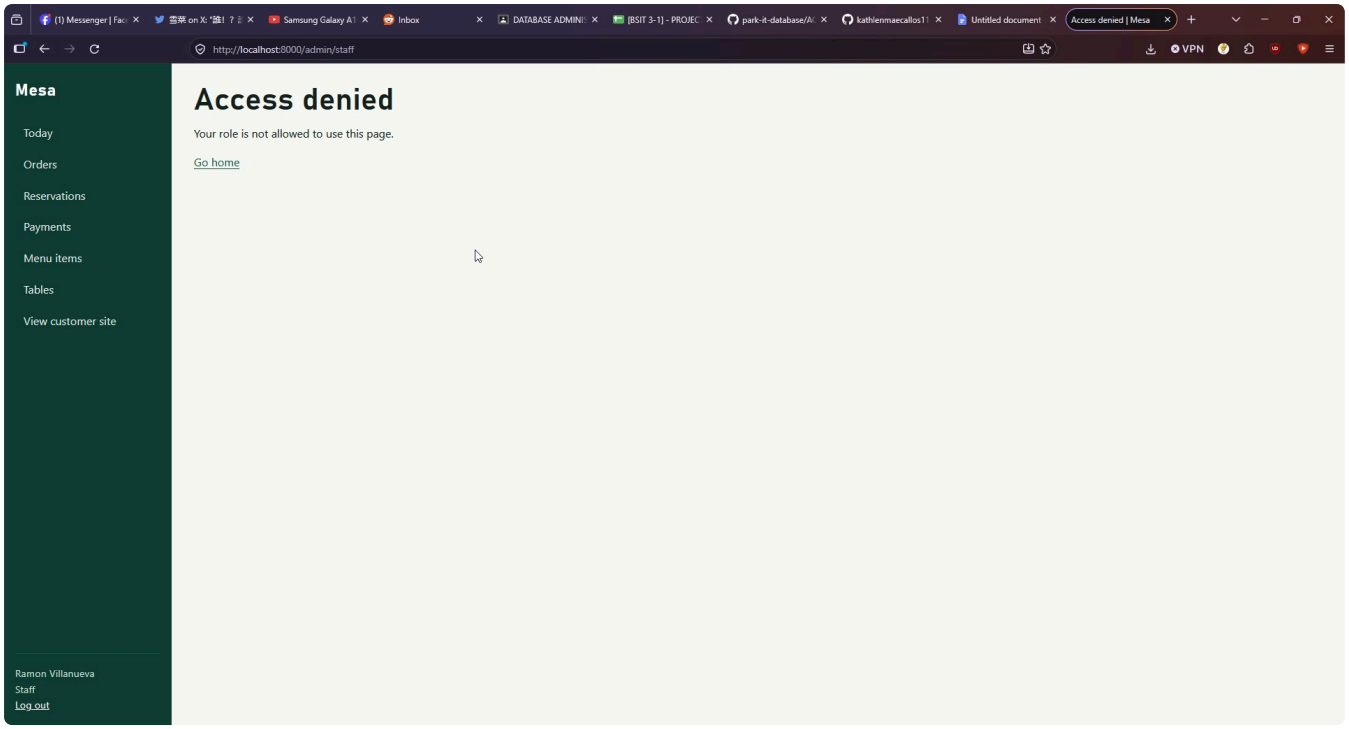
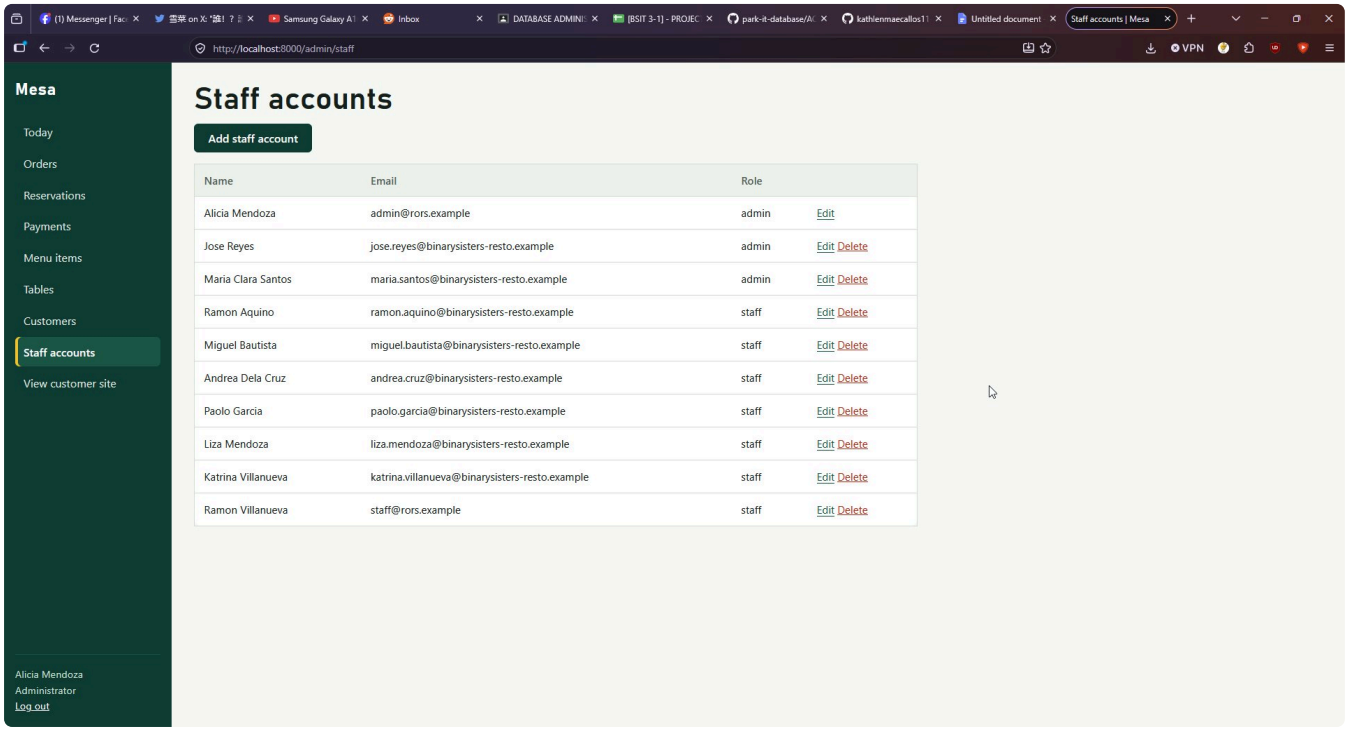



Figure 4. Administrator role opening the Staff accounts page.



# 7. Issues and fixes

| # | Finding | Evidence | Cause and impact | Handling |
|---|---|---|---|---|
| 1 | database/roles/03_set_passwords.psql is missing from the repository | setup.ps1 printed: psql: error: ...03_set_passwords.psql: No such file or directory, then continued | The scripts and README refer to this file. Without it, all five login accounts have no password and rors_app cannot connect, so the application cannot start | Passwords were set manually with ALTER ROLE and kept in a local file outside the repository. Reported to the owners as a missing file |
| 2 | rors_dba holds ALL on every table, including TRUNCATE, REFERENCES, TRIGGER and MAINTAIN, plus CREATE on schema public. It can also read the password hash columns | \dp shows rors_admin_grp=arwdDxtm on every table | Acceptable for a DBA account that is not a superuser, but TRUNCATE on order and payment history is a wider risk than the application needs | Recommendation only: document why the DBA needs TRUNCATE, or remove it and the unneeded rights |
| 3 | rors_report can read customer names, emails and phone numbers through v_customer_directory and v_order_summary | \dp shows rors_readonly_grp=r on the views | Personal data is visible to the reporting role, although password hashes are not | Recommendation only: confirm this exposure is intended for auditors |
| 4 | All three application roles share one database account | Section 3 | The database cannot tell a Customer from an Administrator, so a bug in the PHP authorization checks would be able to use every runtime privilege, including DELETE on staff | Design limitation, noted. Controls are enforced in the application code |

Two expectations did not apply on this setup. First, we expected PUBLIC to allow connections to the postgres database by default, but the connection was denied (tests 25 and 26). No fix was needed. Second, a single-session impersonation test cannot prove login or CONNECT behaviour, which is why the real-login test was added.

# 8. Feedback to owners

Sent to Binary Sisters on 6 November 2026:

- The file database/roles/03_set_passwords.psql is missing from the repository, which breaks setup.ps1 -WithRoles and setup.sh --with-roles and leaves all accounts without passwords.
- The scripts continue after that error instead of stopping, so the missing file is easy to overlook.

- Consider reviewing the rors_dba privileges (TRUNCATE and the rest of ALL) and confirming that rors_report is meant to see customer contact details.
- Positive findings: no superuser, no PUBLIC privileges, history tables protected from DELETE, credential columns hidden from the read-only role, and a working verification script.

## 9. Limitations

- rors_maint, rors_backup and rors_dba were tested for what they must not do, and rors_maint for monitoring. Running VACUUM and ANALYZE and creating a dump are exercised in Activities 5 and 6.
- Roles are cluster-wide. This cluster also holds roles from other projects, which were not assessed here.
- Row-level security is not used by this project and was not evaluated.

## 10. AI usage declaration

- **Tool:** Claude (Anthropic).
- **Tasks:** interpreting the results of the testing.
- **Validation:** every command was run by the group on its own PostgreSQL server, and every expected result was compared with the actual output. The report was checked against the saved output files.
- **Corrections made:** the AI first predicted that rors_app could connect to the postgres database through the default PUBLIC privilege. The real test showed it was denied, and the report records the real result. The AI also first counted 8 sequences, and the inventory showed 7, so the report uses the inventory.